MDS-004 — Typography System

External publication draft

MDS-004 — Typography System

> *Typography is not decoration. It is the voice of the Companion.*

Purpose

The Material System defines how Mosaic physically exists.

The Typography System defines how Mosaic speaks.

Unlike conventional interface typography systems, which primarily optimise for information density, the Mosaic Typography System is designed around companionship.

Typography should communicate:

- confidence
- calmness
- clarity
- editorial quality
- restraint

The objective is not simply readability.

It is effortless understanding.

Relationship to Previous Specifications

```
[text]
Vision

↓

Principles

↓

Mental Model

↓

Interaction

↓

Composition

↓

Design Tokens

↓

Colour

↓

Materials

↓

Typography

↓

Motion

↓

Components
```

Typography consumes:

- Design Tokens
- Composition
- Material Hierarchy

It reinforces:

- hierarchy
- rhythm
- understanding
- atmosphere

Scope

This specification defines:

- Typography Philosophy
- Reading Hierarchy
- Editorial Rhythm
- Type Scales
- Responsive Typography
- Hero Typography
- Reading Density
- Accessibility
- Typography Resolution
- Runtime Typography

This specification intentionally does **not** define:

- Components
- Motion
- Layout
- Colour
- Materials

Those systems work alongside Typography.

Core Question

MDS-004 exists to answer one question.

> ***How should language communicate understanding?***

Not:

> *Which font should we use?*

Typography Statement

Within Mosaic:

> ***Typography should disappear behind understanding.***

Users should remember:

- what they read

Not:

- how the type looked.

Typography Responsibilities

Typography separates into several conceptual systems.

```
[text]
Editorial Hierarchy

↓

Reading Rhythm

↓

Responsive Scaling

↓

Runtime Adaptation

↓

Presentation
```

Each layer contributes one responsibility.

Expected Outcome

After reading MDS-004 contributors should understand:

- how Mosaic uses typography
- why editorial rhythm matters
- how typography supports Composition
- how responsive typography behaves
- how runtime adaptation works
- how accessibility influences type

without discussing specific font families or rendering engines.

Repository Structure

```
[text]
design/

■■■ mds/

    ■■■ MDS-004 Typography System/

        README.md

        00-document-control.md

        01-typography-philosophy.md

        02-editorial-hierarchy.md

        03-type-scale.md

        04-reading-rhythm.md

        05-hero-typography.md

        06-responsive-typography.md

        07-accessibility.md

        08-runtime-resolution.md

        09-platform-typography.md

        10-variable-fonts.md

        11-governance.md

        12-adrs.md

        13-contributor-guidance.md

        glossary.md

        references.md
```

Dependencies

Required reading:

- MDL-001 → MDL-005
- MDS-001 Design Token Architecture
- MDS-002 Colour System
- MDS-003 Material System

Downstream specifications:

- MDS-005 Motion System
- MDS-006 Composition Engine
- MDS-007 Tile Framework
- MDS-008 Component Library

Review Status

Status

Draft

Owner

Lead Design Systems Architect

Next File

`00-document-control.md`

Document Control

Document Information

Property	Value
Document ID	MDS-004
Title	Mosaic Design System — Typography System
Classification	Internal
Status	Draft
Version	0.1
Owner	Lead Design Systems Architect
Parent Specifications	MDL-001 → MDL-005, MDS-001, MDS-002, MDS-003
Repository	`/design/mds/MDS-004 Typography System/`

Purpose

MDS-004 defines the Typography System used throughout Mosaic.

Typography is not treated as visual styling.

It is treated as the primary mechanism through which Mosaic communicates knowledge.

Where the Material System defines:

> *How the interface physically exists.*

Typography defines:

> *How the interface speaks.*

Its responsibility is to make information feel:

- calm
- intelligent
- effortless
- trustworthy
- editorial

The Typography System therefore becomes a fundamental part of the Companion.

Authority

MDS-004 governs:

- Type hierarchy
- Editorial rhythm

- Reading behaviour
- Responsive typography
- Variable font behaviour
- Accessibility
- Runtime typography
- Cross-platform typography

This specification intentionally does **not** govern:

- Colour
- Materials
- Motion
- Components
- Layout

Those systems work alongside Typography.

Relationship To MDS

Typography sits between Materials and Motion.

```
[mermaid]
flowchart TD

Vision
Vision --> Principles
Principles --> MentalModel
MentalModel --> Interaction
Interaction --> Composition
Composition --> Tokens
Tokens --> Colour
Colour --> Materials
Materials --> Typography
Typography --> Motion
Motion --> Components
Components --> Presentation
```

Typography consumes:

- Composition
- Colour
- Materials

It communicates:

- hierarchy
- rhythm
- language
- understanding

Design Intent

Many interface typography systems optimise for:

- density
- efficiency
- information throughput

Mosaic intentionally optimises for:

- comprehension
- calmness
- editorial quality
- companionship

Typography should encourage users to read naturally rather than process interfaces mechanically.

Reader Expectations

Before reading this specification contributors should already understand:

- MDL-001 Vision
- MDL-002 Principles
- MDL-003 Mental Model
- MDL-004 Interaction Model
- MDL-005 Composition Model
- MDS-001 Design Token Architecture
- MDS-002 Colour System
- MDS-003 Material System

Typography builds upon every one of these systems.

It should never redefine them.

Architectural Scope

The Typography System defines:

- editorial hierarchy
- reading rhythm
- semantic typography
- runtime scaling
- accessibility

- responsive typography

It intentionally avoids implementation-specific concerns such as:

- CSS font declarations
- Flutter TextTheme
- SwiftUI Font
- Compose Typography

Those are implementation artefacts generated from this architecture.

Stability

Expected lifetime.

Artefact	Expected Lifetime
Font Implementation	Months
Variable Font Support	Months
Font Family	Years
Typography Hierarchy	Years
Typography Philosophy	Decades

Rendering technology may evolve.

Reading behaviour should remain recognisably Mosaic.

Success Criteria

MDS-004 succeeds when:

- users instinctively know what to read first
- reading feels calm rather than mechanical
- typography supports Composition rather than competing with it
- long-form reading remains comfortable
- interfaces feel editorial rather than technical
- typography quietly disappears behind understanding

Users should remember:

- the story,
- the information,
- the entertainment.

They should rarely remember the typography itself.

Review Status

Status

Draft

Dependencies

- MDL-001 → MDL-005
- MDS-001
- MDS-002
- MDS-003

Supersedes

None.

Next File

`01-typography-philosophy.md`

Typography Philosophy

Purpose

Before defining scales, hierarchy or responsive behaviour, contributors must first understand what typography represents within Mosaic.

Many interface design systems treat typography as a method of displaying information efficiently.

Mosaic intentionally adopts a different philosophy.

Typography exists to communicate understanding with the same calm confidence as a knowledgeable friend.

It should never feel mechanical.

It should never feel corporate.

It should never draw unnecessary attention to itself.

Typography is the voice of the Companion.

Philosophy Statement

> ***Typography should quietly guide understanding while allowing entertainment and ideas to remain the centre of attention.***

Everything within the Typography System derives from this statement.

Typography Is Conversation

The interface should not feel as though it is issuing commands.

It should feel as though it is speaking naturally.

Examples.

Poor.

```
WATCH NOW
```

Preferred.

```
Continue watching
```

Poor.

```
LIBRARY UPDATED
```

Preferred.

```
New episodes are available
```

Typography should communicate with confidence.

Not urgency.

Typography Exists To Be Forgotten

Users should remember:

- what they discovered,
- what they watched,
- what they read,

rather than:

- font weights,
- decorative headings,
- oversized typography.

Typography succeeds when it quietly disappears behind understanding.

Editorial Rather Than Technical

The Mosaic Typography System intentionally takes inspiration from editorial design rather than dashboard software.

Editorial typography encourages:

- reading
- reflection
- pacing
- rhythm

Dashboard typography often encourages:

- scanning
- monitoring
- processing

Entertainment should feel read.

Not processed.

Calm Before Density

Interfaces often attempt to maximise information density.

Typography should instead maximise comprehension.

Sometimes this means:

- shorter line lengths,
- larger breathing spaces,
- quieter hierarchy,

- fewer simultaneous type styles.

Typography should reduce cognitive effort rather than maximise information throughput.

Hierarchy Before Size

Large text is not automatically important.

Hierarchy should emerge from:

- Composition,
- spacing,
- rhythm,
- proximity,
- typography.

Font size is only one tool.

It should not become the primary mechanism communicating importance.

Reading Is A Journey

Typography should encourage continuous reading.

Users should naturally progress from:

```
[text]
Hero

↓

Supporting Information

↓

Context

↓

Detail
```

Every transition should feel inevitable.

Readers should never wonder:

> *"Where should I look next?"*

Confidence

Typography should feel confident.

Confidence is communicated through:

- restraint,

- consistency,
- predictable hierarchy,
- generous rhythm.

Not through:

- oversized headings,
- decorative fonts,
- excessive weight,
- dramatic styling.

The interface should never shout.

Warmth

Although Typography should remain professional, it should never feel cold.

The Companion should sound:

- knowledgeable,
- thoughtful,
- reassuring,
- quietly enthusiastic.

The interface should avoid:

- robotic language,
- unnecessary technical jargon,
- marketing exaggeration,
- aggressive calls to action.

Users should feel they are being guided.

Not sold to.

Typography Supports Materials

Typography exists inside Materials.

It should therefore respect them.

Examples.

Hero Material.

↓

Slightly more spacious typography.

Overlay Material.

↓

Higher clarity.

Canvas.

↓

Editorial rhythm.

Materials establish environment.

Typography establishes understanding.

The two systems should always feel related.

Typography Supports Atmosphere

Runtime Atmosphere may subtly influence the emotional character of typography.

It should never reduce:

- readability,
- contrast,
- hierarchy.

Atmosphere influences surrounding materials.

Typography remains comparatively stable.

Words should always remain the clearest part of the interface.

Typography Supports Composition

Typography should reinforce the Composition Model.

Examples.

Primary understanding.

↓

Highest typographic emphasis.

Supporting understanding.

↓

Moderate emphasis.

Peripheral information.

↓

Quiet typography.

Typography should follow Composition.

It should never compete with it.

Typography Across Domains

The same Typography System should support:

- films,
- television,
- books,
- music,
- administration.

Different domains may alter rhythm.

They should never invent new typographic languages.

Users should always feel:

> *This is still Mosaic.*

Accessibility

Typography should remain understandable before colour is applied.

Removing:

- colour,
- materials,
- atmosphere,

should still preserve:

- hierarchy,
- rhythm,
- comprehension.

Typography therefore becomes one of the strongest accessibility mechanisms within the Design System.

Good Examples

Film

Large but restrained title.

Comfortable supporting metadata.

Editorial spacing.

The interface feels cinematic without becoming theatrical.

Reading

Long-form typography.

Generous rhythm.

Minimal interruption.

The interface quietly supports extended reading sessions.

Administration

Clear structure.

Higher information density.

Editorial hierarchy preserved.

The interface becomes more functional without abandoning the Mosaic voice.

Anti-patterns

Dashboard Typography

Every piece of information competes equally.

Scanning replaces reading.

Marketing Typography

Oversized headings.

Excessive emphasis.

Promotional tone.

The interface begins selling rather than accompanying.

Decorative Typography

Typography becomes a visual feature.

Understanding weakens.

Dense Typography

Small text.

Minimal spacing.

Continuous visual pressure.

Reading becomes work.

Typography Philosophy Model

```
[mermaid]
flowchart TD

Composition
Composition --> Typography
Typography --> ReadingRhythm
ReadingRhythm --> Understanding
Understanding --> Entertainment
```

Typography exists to strengthen understanding.

Entertainment remains the destination.

Relationship To Future Chapters

The remaining chapters formalise this philosophy into implementation.

They define:

- Editorial Hierarchy
- Type Scale
- Reading Rhythm
- Hero Typography
- Responsive Typography
- Accessibility
- Runtime Resolution

Every implementation decision should reinforce the philosophy established here.

Summary

Typography is the voice of Mosaic.

It should feel:

- calm,
- intelligent,
- editorial,
- trustworthy,
- quietly human.

When successful, users should never think about typography.

They should simply feel that understanding came effortlessly.

That quiet confidence is the defining characteristic of the Mosaic Typography System.

Review Status

Status

Draft

Next File

`02-editorial-hierarchy.md`

Editorial Hierarchy

Purpose

Typography communicates more than language.

It communicates:

- importance,
- confidence,
- rhythm,
- intent.

Within Mosaic this communication is intentionally modelled after editorial design rather than dashboard software.

Editorial Hierarchy defines how typography guides attention through a Composition.

It answers one question.

> ***"What should the reader understand first?"***

Definition

Within MDS, **Editorial Hierarchy** is defined as:

> ***The intentional organisation of written language into a reading experience that mirrors the conceptual hierarchy established by the Composition Model.***

Editorial Hierarchy should never create hierarchy independently.

It reinforces hierarchy that already exists.

Why Editorial?

Entertainment is consumed.

Not managed.

People naturally expect:

- stories,
- books,
- magazines,
- albums,
- films

to present information editorially.

Traditional software instead presents information transactionally.

Examples.

```
Runtime

Genre

Rating

Studio

Release
```

Everything receives identical emphasis.

Editorial Hierarchy instead tells a story.

```
Current Focus

↓

Current Progress

↓

Supporting Information

↓

Additional Context
```

The user reads naturally.

Rather than scanning mechanically.

Composition Leads

Typography should always follow Composition.

Never the reverse.

Conceptually.

```
[text]
Composition

↓

Editorial Hierarchy

↓

Typography

↓

Presentation
```

The Composition determines:

- what matters.

Typography determines:

- how that importance is communicated.

Reading Order

Every Composition should establish one natural reading path.

Typical order.

```
[text]
Hero

↓

Primary Action

↓

Supporting Information

↓

Context

↓

Relationships

↓

Peripheral Information
```

Readers should never consciously search for the next piece of information.

Editorial Hierarchy should quietly guide them.

Hierarchy Levels

The Typography System defines six editorial levels.

```
[text]
Display

↓

Heading

↓

Section

↓

Body

↓

Supporting

↓

Caption
```

Each level exists for one purpose.

None should duplicate another.

Display

Purpose.

Communicate the Hero.

Display typography should appear rarely.

Normally only:

- Hero titles,
- onboarding,
- major transitions,
- immersive artwork.

Display typography should never become routine.

Its rarity preserves its impact.

Heading

Purpose.

Introduce major concepts.

Examples.

- Film title
- Book title
- Artist
- Collection

Headings establish orientation.

Not decoration.

Section

Purpose.

Organise understanding.

Examples.

- Continue Watching
- Related Works
- Chapters
- Cast

Section typography should quietly structure the Composition.

Body

Purpose.

Communicate primary reading content.

Examples.

- descriptions
- metadata
- reviews
- summaries

Body typography should prioritise:

- comfort,
- rhythm,
- clarity.

Body text represents the majority of reading within Mosaic.

Supporting

Purpose.

Communicate secondary understanding.

Examples.

- runtime,
- publication date,
- codec,
- subtitle language,
- metadata.

Supporting typography should remain readable without competing with primary information.

Caption

Purpose.

Communicate quiet context.

Examples.

- timestamps,
- file size,
- diagnostics,

- technical metadata.

Caption typography should remain present without drawing unnecessary attention.

Hierarchy Is Relative

Editorial Hierarchy depends upon Context.

Example.

Current Context.

```
Watching
```

Episode title becomes:

Heading.

Runtime becomes:

Supporting.

Later.

```
Browsing Reviews
```

Review title becomes:

Heading.

Episode runtime becomes:

Caption.

Typography adapts because understanding changes.

Hierarchy And Space

Editorial Hierarchy should work together with Breathing Space.

Example.

```
Heading
```

↓

```
Space
```

↓

```
Body
```

↓

```
Space
```

↓

```
Supporting
```

Whitespace communicates editorial rhythm.

Typography alone should never carry the full burden of hierarchy.

Hero Typography

The Hero should receive the strongest editorial treatment.

However...

It should not become theatrical.

Avoid:

- oversized typography,
- excessive weight,
- unnecessary visual effects.

The Hero should communicate quiet confidence.

Not spectacle.

Editorial Consistency

Every Domain should reuse the same editorial hierarchy.

Television.

↓

Episode title.

Books.

↓

Chapter title.

Music.

↓

Track title.

Different content.

Identical editorial language.

Users should never learn multiple typographic systems.

Runtime Adaptation

Editorial Hierarchy remains conceptually stable.

Runtime may adapt:

- scale,
- spacing,
- weight,

- contrast.

Meaning remains unchanged.

The same Heading should remain a Heading regardless of:

- Light Mode,
- Dark Mode,
- Television,
- Mobile.

Accessibility

Editorial Hierarchy should survive:

- reduced colour,
- reduced motion,
- high contrast,
- enlarged text.

If hierarchy disappears when colour is removed...

Typography has failed.

Materials

Typography should respond to surrounding materials.

Hero Material.

↓

More generous spacing.

Overlay Material.

↓

Higher clarity.

Canvas.

↓

Editorial calmness.

Materials influence typography subtly.

They never redefine hierarchy.

Good Examples

Film

Display.

↓

Film title.

Heading.

↓

Continue Watching.

Body.

↓

Synopsis.

Supporting.

↓

Runtime.

Caption.

↓

Codec.

The reader instinctively understands where to begin.

Reading

Heading.

↓

Book title.

Section.

↓

Current Chapter.

Body.

↓

Summary.

Supporting.

↓

Progress.

The hierarchy quietly encourages continued reading.

Administration

Heading.

↓

Users.

Section.

↓

Permissions.

Body.

↓

Configuration.

Caption.

↓

Diagnostics.

Editorial rhythm remains recognisably Mosaic despite increased information density.

Anti-patterns

Equal Typography

Every piece of text appears identical.

Users construct hierarchy themselves.

Decorative Hierarchy

Hierarchy depends upon colour or animation.

Typography should remain understandable independently.

Oversized Headlines

Every heading competes for attention.

The interface begins shouting.

Dense Metadata

Supporting information visually overwhelms primary content.

The Composition weakens.

Editorial Hierarchy Model

```
[mermaid]
flowchart TD

Composition
Composition --> EditorialHierarchy
EditorialHierarchy --> TypographyScale
TypographyScale --> ReadingRhythm
ReadingRhythm --> Presentation
```

Editorial Hierarchy transforms conceptual importance into readable structure.

Relationship To Future Chapters

The next chapter defines the **Type Scale**.

Editorial Hierarchy answers:

> ***What should be emphasised?***

The Type Scale answers:

> ***How should that emphasis be physically expressed?***

Together they establish the typographic language of Mosaic.

Summary

Editorial Hierarchy is the bridge between Composition and Typography.

It allows users to instinctively understand:

- what matters,
- what supports it,
- what can wait.

Typography should therefore feel less like interface chrome...

...and more like a carefully edited publication.

That editorial quality is one of the defining characteristics of the Mosaic Design System.

Review Status

Status

Draft

Next File

`03-type-scale.md`

Type Scale

Purpose

Editorial Hierarchy defines **what deserves emphasis**.

The Type Scale defines **how that emphasis is physically communicated**.

Unlike many design systems, the Mosaic Type Scale is **not** a mathematical ladder of font sizes.

It is a reading system.

Every step exists to support:

- understanding,
- rhythm,
- hierarchy,
- companionship.

Typography should encourage reading.

Not simply display information.

Definition

Within MDS, the **Type Scale** is defined as:

> ***The ordered system of typographic roles through which editorial hierarchy becomes physically readable.***

The Type Scale expresses hierarchy.

It does not create it.

Hierarchy already exists within the Composition.

Philosophy

Many interfaces define typography like this.

```
[text]
12

14

16

18

20

24

32
```

```
48
```

These are numbers.

They communicate almost nothing.

Mosaic instead thinks in reading roles.

```
[text]
Display

↓

Heading

↓

Section

↓

Body

↓

Supporting

↓

Caption
```

Values exist only to support those roles.

Scale Before Size

Typography should always be selected by role.

Not measurement.

Incorrect.

```
[text]
Use 24px
```

Correct.

```
[text]
Use Heading
```

The Design System determines the correct implementation.

Applications should never depend upon physical measurements.

Scale Hierarchy

The Mosaic Type Scale contains six primary levels.

```
[text]
Display

↓

Heading

↓
```

```
Section

↓

Body

↓

Supporting

↓

Caption
```

Each level exists because it communicates a different editorial responsibility.

Display

Purpose.

Communicate major emotional moments.

Examples include:

- Hero title
- onboarding
- welcome
- major transitions

Display typography should remain rare.

Overuse weakens emphasis.

Heading

Purpose.

Establish orientation.

Examples.

- film title
- book title
- artist
- collection

Headings should communicate confidence.

Not visual drama.

Section

Purpose.

Organise the Composition.

Examples.

- Continue Watching
- Related Works
- Recently Added
- Chapters

Sections divide understanding into meaningful editorial groups.

Body

Purpose.

Support sustained reading.

Body typography represents the majority of reading throughout Mosaic.

Examples.

- synopsis
- descriptions
- reviews
- biographies

Comfort should always have higher priority than compactness.

Supporting

Purpose.

Communicate secondary understanding.

Examples.

- runtime
- release year
- author
- subtitle language
- resolution

Supporting typography should remain easy to read while remaining visually quieter than Body text.

Caption

Purpose.

Communicate peripheral detail.

Examples.

- timestamps
- codec
- bitrate
- diagnostics
- technical metadata

Caption typography should never become unreadable.

Quiet does not mean small.

It means lower editorial importance.

Scale Relationships

Each level should feel related to the levels around it.

Example.

```text
Heading

↓

Section

↓

Body
```

The transition should feel gradual.

Not abrupt.

Readers should naturally move through the hierarchy without consciously noticing changes in scale.

Optical Rhythm

Typography should create rhythm rather than mathematical progression.

Poor.

```text
16

18

20

22

24
```

Preferred.

```text
Display

↓
```

```
Heading
↓
Section
↓
Body
```

Readers perceive rhythm.

Not pixel values.

The scale should therefore optimise perception.

Not arithmetic.

Reading Distance

Different contexts require different perceived scale.

Examples.

Television.

↓

Greater viewing distance.

↓

Larger physical implementation.

Phone.

↓

Closer viewing distance.

↓

Smaller physical implementation.

The editorial role remains identical.

Only implementation changes.

Hero Scale

Hero Typography should possess the greatest physical presence.

However...

It should never dominate through size alone.

Hero emphasis should emerge through:

- hierarchy
- composition

- spacing
- materials
- typography

Working together.

Typography should never carry hierarchy independently.

Long-Form Reading

Books.

Reviews.

Descriptions.

Editorial content.

These experiences should optimise:

- line length
- rhythm
- paragraph spacing
- eye movement

The Body role therefore becomes one of the most carefully tuned parts of the entire Typography System.

Comfort should remain the primary objective.

Runtime Adaptation

The Type Scale may adapt according to:

- accessibility
- viewing distance
- platform
- display density

It should never adapt because:

- artwork changed
- atmosphere changed

Atmosphere influences Materials.

Typography remains comparatively stable.

Variable Fonts

Future implementations may use variable font technology.

The Type Scale should express:

- weight
- width
- optical size

through semantic roles.

Applications should never manipulate these values directly.

The Variable Font System is defined later within this specification.

Accessibility

The Type Scale should remain recognisable regardless of:

- increased text size
- reduced vision
- high contrast
- platform scaling

Increasing text size should preserve hierarchy.

Not flatten it.

Every editorial role should remain identifiable.

Good Examples

Film

Display.

↓

Film Title.

Heading.

↓

Continue Watching.

Section.

↓

Cast.

Body.

↓

Synopsis.

Supporting.

↓

Runtime.

Caption.

↓

HDR.

Readers instinctively understand importance.

Reading

Heading.

↓

Book Title.

Section.

↓

Current Chapter.

Body.

↓

Content.

Supporting.

↓

Progress.

Caption.

↓

Page Count.

Long-form reading remains comfortable.

Administration

Heading.

↓

Users.

Section.

↓

Permissions.

Body.

↓

Configuration.

Caption.

↓

Diagnostics.

Editorial consistency remains intact despite higher information density.

Anti-patterns

Pixel Thinking

Selecting typography by measurement.

Decorative Scale

Large text used purely because space exists.

Tiny Metadata

Caption becoming unreadable.

Equal Scale

Every editorial role appearing visually identical.

Hierarchy disappears.

Type Scale Model

```
[mermaid]
flowchart TD

EditorialHierarchy
EditorialHierarchy --> TypeScale
TypeScale --> ResponsiveScaling
ResponsiveScaling --> Presentation
```

Editorial intent determines the role.

The Type Scale determines the physical expression.

Relationship To Future Chapters

The next chapter defines **Reading Rhythm**.

The Type Scale answers:

> ***How large should each editorial role feel?***

Reading Rhythm answers:

> ***How should those roles work together over time?***

Together they create typography that feels read rather than scanned.

Summary

The Type Scale is not a collection of font sizes.

It is a reading hierarchy.

Every level should encourage effortless movement through the user's current World.

Users should naturally know:

- where to begin,
- what deserves attention,
- what supports understanding,

without consciously analysing typography.

That effortless progression is the defining objective of the Mosaic Type Scale.

Review Status

Status

Draft

Next File

`04-reading-rhythm.md`

Reading Rhythm

Purpose

Typography is more than letters.

It is movement.

Readers do not consume interfaces one word at a time.

They move through ideas.

Reading Rhythm defines how typography guides that movement.

Unlike many interface systems, Mosaic intentionally optimises for continuous reading rather than rapid scanning.

Reading should feel:

- natural,
- calm,
- uninterrupted,
- inevitable.

The Typography System therefore treats rhythm as an architectural concern rather than a visual preference.

Definition

Within MDS, **Reading Rhythm** is defined as:

> ***The deliberate pacing of typography, spacing and hierarchy that guides readers naturally through a Composition.***

Reading Rhythm exists between typography and composition.

It transforms information into an effortless reading experience.

Philosophy

A good novel possesses rhythm.

A well-designed magazine possesses rhythm.

A thoughtful article possesses rhythm.

Readers rarely notice it consciously.

They simply continue reading.

The same principle should apply to Mosaic.

Users should rarely stop to determine:

- where to read next,

- whether something is important,
- whether they missed information.

The interface should already have answered those questions.

Reading Before Scanning

Many productivity interfaces optimise for scanning.

Entertainment interfaces should optimise for reading.

Scanning.

```
[text]
Metric

Metric

Metric

Metric
```

Reading.

```
[text]
Current Focus

↓

Progress

↓

Supporting Context

↓

Related Discovery
```

The second encourages understanding rather than information processing.

Rhythm Emerges From Composition

Reading Rhythm should never exist independently.

It follows:

```
[text]
Composition

↓

Editorial Hierarchy

↓

Reading Rhythm

↓

Presentation
```

Composition establishes understanding.

Reading Rhythm determines how that understanding unfolds.

Vertical Flow

Readers naturally move vertically.

The Composition should support this behaviour.

Preferred.

```text
Hero

↓

Primary Action

↓

Supporting Context

↓

Related Information
```

Avoid fragmented reading paths requiring constant visual searching.

The eye should move comfortably through the Composition.

Line Length

Reading Rhythm depends heavily upon comfortable line lengths.

Long-form reading should avoid:

- excessively long lines,
- extremely narrow columns,
- inconsistent wrapping.

Future implementations should optimise line length according to:

- device,
- viewing distance,
- reading context.

Comfort always possesses higher priority than fitting additional information onto the screen.

Paragraph Rhythm

Paragraphs communicate pace.

Large uninterrupted blocks increase cognitive effort.

Very short fragmented sentences weaken narrative flow.

Descriptions.

Reviews.

Editorial content.

All should possess consistent paragraph rhythm.

The interface should feel composed.

Not assembled.

Spacing Rhythm

Whitespace is part of typography.

Conceptually.

```text
Heading

↓

Space

↓

Body

↓

Space

↓

Supporting
```

Spacing communicates pauses.

These pauses improve understanding.

Removing them increases cognitive load.

Tempo

Different activities require different reading tempo.

Watching.

↓

Very little reading.

Reading.

↓

Sustained reading.

Administration.

↓

Faster information processing.

The Typography System should naturally adapt rhythm without changing its underlying editorial language.

Hero Rhythm

Hero Typography should breathe.

Examples.

Large title.

↓

Generous spacing.

↓

Supporting metadata.

↓

Primary action.

Nothing should feel hurried.

The Hero introduces the experience.

It should feel confident.

Metadata Rhythm

Metadata should never interrupt primary reading.

Poor.

```
[text]
Title

Runtime

Codec

Resolution

Studio

Rating

Description
```

Preferred.

```
[text]
Title

↓

Description

↓

Supporting Metadata
```

Supporting information should remain available without constantly demanding attention.

Rhythm Across Domains

Television.

↓

Progressive reading.

Books.

↓

Long-form rhythm.

Music.

↓

Compact rhythm.

Administration.

↓

Higher information density.

Although tempo changes...

The editorial language should remain recognisably Mosaic.

Rhythm Across Devices

Desktop.

↓

Longer visual rhythm.

Tablet.

↓

Editorial rhythm.

Phone.

↓

Shorter but equivalent rhythm.

Television.

↓

Larger pauses.

Greater viewing distance.

The physical implementation changes.

The perceived rhythm should remain constant.

Runtime Adaptation

Reading Rhythm should remain remarkably stable.

Runtime may adjust:

- spacing,
- scale,
- line length,

according to:

- accessibility,
- device,
- viewing distance.

Artwork should never directly alter reading rhythm.

Atmosphere belongs to materials.

Rhythm belongs to typography.

Accessibility

Accessibility should strengthen rhythm.

Examples.

Larger text.

↓

Larger spacing.

Higher contrast.

↓

Clearer hierarchy.

Reading should become easier.

Never denser.

Materials

Typography should appear physically related to surrounding materials.

Hero Material.

↓

More breathing space.

Overlay Material.

↓

Greater clarity.

Canvas.

↓

Editorial calmness.

Materials provide environment.

Typography provides cadence.

Together they create one coherent experience.

Good Examples

Film

Title.

↓

Synopsis.

↓

Continue Watching.

↓

Supporting metadata.

The eye naturally progresses through the experience.

Book

Book title.

↓

Current chapter.

↓

Reading summary.

↓

Bookmarks.

↓

Technical information.

Readers remain immersed.

Administration

Users.

↓

Permissions.

↓

Configuration.

↓

Diagnostics.

Higher density.

Identical editorial rhythm.

Anti-patterns

Dashboard Rhythm

Everything presented simultaneously.

The eye constantly searches.

Interrupted Reading

Metadata repeatedly interrupts narrative information.

Compressed Typography

Paragraphs and headings visually collapse together.

Rhythm disappears.

Decorative Spacing

Whitespace exists without editorial purpose.

Rhythm should always communicate understanding.

Reading Rhythm Model

```
[mermaid]
flowchart TD

Composition
Composition --> EditorialHierarchy
EditorialHierarchy --> ReadingRhythm
ReadingRhythm --> Typography
Typography --> Understanding
```

Rhythm transforms typography into an effortless reading experience.

Relationship To Future Chapters

The next chapter defines **Hero Typography**.

Reading Rhythm explains:

> ***How readers move through information.***

Hero Typography explains:

> ***How the first words they encounter establish the emotional tone of the experience.***

Together they establish the voice of Mosaic.

Summary

Reading Rhythm is one of the quietest systems within Mosaic.

Users should rarely notice it consciously.

Instead they should simply continue reading.

The interface should feel less like software...

...and more like a thoughtfully edited publication.

When Reading Rhythm succeeds, understanding becomes almost effortless.

Review Status

Status

Draft

Next File

`05-hero-typography.md`

Hero Typography

Purpose

The Hero represents the centre of the current Composition.

Hero Typography represents the voice of that Hero.

Unlike marketing interfaces, which often use oversized typography to dominate attention, Mosaic treats Hero Typography with restraint.

Its purpose is not to impress.

Its purpose is to orient.

The Hero should quietly answer:

- What am I looking at?
- Why does it matter?
- What should I do next?

before the user consciously asks those questions.

Definition

Within MDS, **Hero Typography** is defined as:

> ***The highest level of editorial typography used to establish the current Focus while preserving the emotional dominance of the user's entertainment.***

Hero Typography communicates:

- identity,
- confidence,
- orientation,

rather than spectacle.

Philosophy

The Hero should never feel like advertising.

Instead imagine opening a beautifully designed hardback book.

The title is confident.

Comfortable.

Elegant.

It does not shout.

Hero Typography should evoke that same feeling.

The interface should introduce entertainment with quiet confidence.

One Hero Voice

Every Composition should normally possess one Hero voice.

Poor.

```
Featured

Trending

Recommended

Continue Watching
```

Every heading competes.

Preferred.

```
Continue Watching

↓

Frieren

↓

Episode 18
```

One clear editorial voice.

Everything else supports it.

Hero Before Decoration

Hero Typography should communicate importance before visual style.

Avoid relying upon:

- oversized text,
- excessive weight,
- bright colours,
- dramatic effects.

Importance should already exist within:

- Composition,
- Material,
- Hierarchy.

Typography simply reinforces it.

Hero And Artwork

Artwork remains the emotional centre.

Hero Typography provides context.

Relationship.

```text
Artwork

↓

Hero Typography

↓

Supporting Information
```

Typography should never obscure artwork.

Nor should artwork reduce typographic clarity.

Both systems should feel deliberately balanced.

Hero Rhythm

Hero Typography establishes the editorial rhythm for the entire Composition.

Typical progression.

```text
Hero Title

↓

Supporting Metadata

↓

Primary Action

↓

Context

↓

Related Information
```

Readers should instinctively understand where to continue.

The Hero begins the conversation.

The remainder of the Composition continues it.

Hero Weight

Hero Typography should communicate confidence through restraint.

Preferred characteristics.

- moderate weight,
- generous spacing,
- balanced proportions,
- comfortable line length.

Avoid:

- extremely bold weights,
- condensed typography,
- excessive uppercase,
- promotional emphasis.

Entertainment already provides emotional intensity.

Typography should remain composed.

Hero Length

Hero Typography should gracefully support varying title lengths.

Examples.

```
Up
```

```
The Lord of the Rings:
The Return of the King
```

```
Frieren:
Beyond Journey's End
```

The editorial hierarchy should remain consistent regardless of title complexity.

Future implementations should optimise:

- wrapping,
- line balancing,
- optical alignment.

Hero Metadata

Supporting metadata should remain visually subordinate.

Examples include:

- release year,
- runtime,
- author,
- studio,
- rating.

Incorrect.

```
[text]
Title

Runtime

Codec

Audio

Resolution

HDR

Dolby Vision
```

Preferred.

```
[text]
Title

↓

Synopsis

↓

Supporting Metadata
```

Technical information should never interrupt editorial flow.

Hero Across Domains

Television.

↓

Series.

↓

Episode.

Books.

↓

Book.

↓

Current Chapter.

Music.

↓

Album.

↓

Current Track.

Administration.

↓

Current Task.

The editorial relationship remains identical.

Only the content changes.

Hero Across Themes

Light Theme.

Typography should feel:

- editorial,
- airy,
- quietly premium.

Dark Theme.

Typography should feel:

- cinematic,
- confident,
- softly illuminated.

The typographic identity remains identical.

Only environmental interpretation changes.

Hero Across Devices

Desktop.

↓

Expanded hierarchy.

Tablet.

↓

Editorial hierarchy.

Phone.

↓

Compact hierarchy.

Television.

↓

Greater physical scale.

Greater viewing distance.

The Hero should remain immediately recognisable regardless of implementation.

Runtime Behaviour

Hero Typography should remain remarkably stable.

Runtime may adjust:

- contrast,
- scale,
- spacing,
- optical sizing.

Runtime should never dramatically alter:

- hierarchy,
- weight,
- editorial tone.

The Hero should always sound like the same Companion.

Accessibility

Hero Typography should remain identifiable under:

- enlarged text,
- reduced colour,
- high contrast,
- reduced motion.

Accessibility should preserve:

- hierarchy,
- rhythm,
- confidence.

Hero Typography should never become promotional simply because text becomes larger.

Materials

Hero Typography should respect surrounding materials.

Hero Acrylic.

↓

More breathing space.

↓

Softer atmosphere.

↓

Higher perceived refinement.

Typography and Materials should feel like one physical system.

Neither should dominate the other.

Good Examples

Film

Film title.

↓

Synopsis.

↓

Continue Watching.

↓

Supporting metadata.

The interface quietly introduces the experience.

Book

Book title.

↓

Current Chapter.

↓

Reading Progress.

↓

Supporting Information.

Editorial rhythm remains calm.

Music

Album.

↓

Artist.

↓

Playback.

↓

Related Albums.

The Hero introduces listening without interrupting it.

Anti-patterns

Marketing Hero

Huge typography competing with artwork.

Promotional Tone

Multiple calls-to-action surrounding the Hero.

Decorative Fonts

Typography becoming visual identity.

Metadata First

Technical information appearing before understanding.

Hero Typography Model

```
[mermaid]
flowchart TD

Hero
Hero --> HeroTypography
HeroTypography --> EditorialHierarchy
EditorialHierarchy --> ReadingRhythm
ReadingRhythm --> Understanding
```

Hero Typography begins the editorial conversation.

The Composition completes it.

Relationship To Future Chapters

The next chapter defines **Responsive Typography**.

Hero Typography explains:

> ***How the Hero speaks.***

Responsive Typography explains:

> ***How that voice remains consistent across every device and viewing environment.***

Together they ensure that the Companion sounds recognisably like Mosaic everywhere.

Summary

Hero Typography should feel like the opening sentence of a beautifully written story.

Confident.

Calm.

Inviting.

It should introduce the user's current World without demanding attention for itself.

The entertainment remains the emotional centre.

Hero Typography simply gives it a voice.

Review Status

Status

Draft

Next File

`06-responsive-typography.md`

Responsive Typography

Purpose

Typography should remain recognisably Mosaic regardless of:

- device,
- display size,
- viewing distance,
- orientation,
- accessibility settings.

Responsive Typography ensures that editorial understanding remains stable while physical implementation adapts.

Unlike many responsive systems, Mosaic does **not** begin with screen width.

It begins with reading.

The objective is not fitting more text onto a display.

The objective is preserving effortless understanding.

Definition

Within MDS, **Responsive Typography** is defined as:

> ***The adaptive implementation of editorial typography across different viewing environments while preserving semantic hierarchy, rhythm and readability.***

Responsive Typography changes implementation.

It never changes editorial meaning.

Philosophy

Responsive typography should answer:

> ***How should this feel to read here?***

Not:

> ***How many pixels are available?***

The Typography System should adapt to the reader.

Not force the reader to adapt to the device.

Editorial Consistency

The following should remain identical across every device.

- Editorial hierarchy
- Reading rhythm
- Semantic roles
- Hero importance
- Supporting information

Only physical implementation changes.

Conceptually.

```
[text]
Heading

↓

Desktop

Heading

↓

Tablet

Heading

↓

Phone

Heading

↓

Television
```

One editorial role.

Many implementations.

Reading Distance

Reading distance is more important than screen size.

Approximate examples.

Device	Reading Distance
Phone	Very Near
Tablet	Near
Desktop	Medium
Television	Far

Typography should adapt according to perceived distance rather than pixel dimensions.

Device Classes

Responsive Typography currently recognises several conceptual environments.

Phone

Characteristics.

- one-handed interaction
- short reading bursts
- interruptions
- close viewing

Typography should favour:

- shorter line lengths
- stronger hierarchy
- comfortable touch reading

Tablet

Characteristics.

- immersive reading
- medium distance
- portrait usage
- editorial layouts

Tablet typography should closely resemble printed editorial material.

Desktop

Characteristics.

- multitasking
- larger information density
- longer sessions

Typography should remain calm despite additional available space.

Extra space should improve rhythm.

Not increase clutter.

Television

Characteristics.

- greater viewing distance

- remote interaction
- cinematic presentation

Typography should become physically larger.

It should not become visually louder.

Responsive Scale

Every editorial role should scale proportionally.

Conceptually.

```text
Display

↓

Heading

↓

Section

↓

Body

↓

Supporting

↓

Caption
```

The relationships between roles should remain stable regardless of implementation.

Scaling should preserve hierarchy.

Not flatten it.

Line Length

Comfortable line length is one of the strongest determinants of readability.

Future implementations should optimise line length according to:

- device
- orientation
- reading context

Long-form reading should remain comfortable on every supported platform.

Responsive Rhythm

Responsive Typography should preserve editorial rhythm.

Desktop.

↓

Longer pauses.

Phone.

↓

Shorter pauses.

Television.

↓

Larger physical spacing.

Although measurements differ...

Readers should perceive identical rhythm.

Orientation

Orientation should influence implementation.

Not hierarchy.

Portrait.

↓

Narrower reading measure.

Landscape.

↓

Wider reading measure.

The editorial structure should remain unchanged.

Runtime Adaptation

Responsive Typography may adapt according to:

- accessibility
- display density
- viewing distance
- operating system scaling

It should **not** adapt according to:

- artwork
- runtime atmosphere
- entertainment genre

Typography remains comparatively stable.

Materials carry emotional adaptation.

Variable Fonts

Future implementations should preferentially use variable font technology.

Responsive behaviour may therefore adapt:

- optical size
- width
- weight

while preserving the same editorial role.

Applications should never manipulate these properties directly.

The Typography Resolver owns these decisions.

Accessibility

Accessibility should strengthen Responsive Typography.

Examples.

Large text.

↓

Preserve hierarchy.

Higher contrast.

↓

Improve readability.

Reduced vision.

↓

Increase spacing.

Responsiveness should improve understanding rather than merely enlarging text.

Hero Behaviour

Hero Typography should remain recognisable across every environment.

Desktop.

↓

Large editorial Hero.

Phone.

↓

Compact editorial Hero.

Television.

↓

Large cinematic Hero.

The Hero should always feel like the same voice.

Only its physical expression changes.

Long-Form Reading

Books.

Reviews.

Descriptions.

Long-form editorial content should receive special responsive treatment.

Examples include:

- comfortable measure
- increased line spacing
- generous paragraph rhythm
- stable hierarchy

Reading should remain enjoyable rather than efficient.

Cross-Platform Consistency

Future Mosaic clients should share one typographic language.

Web.

Flutter.

SwiftUI.

Compose.

Desktop.

Television.

Each client may implement typography differently.

Readers should experience one editorial voice.

Plugins

Extensions should never define responsive typography.

Plugins contribute:

- information
- relationships
- editorial content

The Typography System determines:

- hierarchy
- scaling
- rhythm
- implementation

Every extension therefore automatically inherits future typographic improvements.

Good Examples

Phone

Compact hierarchy.

Comfortable reading.

Shorter line lengths.

Editorial rhythm preserved.

Desktop

Greater breathing space.

Longer paragraphs.

Stable editorial pacing.

The interface feels spacious rather than empty.

Television

Large typography.

Long viewing distance.

Strong readability.

Atmosphere remains secondary to comprehension.

Anti-patterns

Breakpoint Typography

Typography changing purely because viewport width crossed a breakpoint.

Device Hierarchy

Heading becoming Body on smaller devices.

Editorial meaning has been lost.

Dense Desktop

Using extra space to display more text rather than improve rhythm.

Tiny Mobile Text

Shrinking typography to fit more information.

Reading becomes work.

Responsive Typography Model

```
[mermaid]
flowchart TD

EditorialHierarchy
EditorialHierarchy --> ResponsiveTypography
ResponsiveTypography --> ViewingEnvironment
ViewingEnvironment --> TypographyResolver
TypographyResolver --> Presentation
```

Responsive Typography preserves understanding while adapting physical implementation.

Relationship To Future Chapters

The next chapter defines **Accessibility**.

Responsive Typography explains:

> ***How typography adapts to different environments.***

Accessibility explains:

> ***How typography adapts to different people.***

Together they ensure the Companion remains understandable for every reader.

Summary

Responsive Typography ensures that Mosaic speaks with one consistent editorial voice across every supported platform.

Different devices should never produce different typography philosophies.

They should simply produce different physical expressions of the same language.

Readers should always feel that:

> *The Companion is speaking to me.*

Not:

> *This is the mobile version.*

That consistency is one of the defining goals of the Mosaic Typography System.

Review Status

Status

Draft

Next File

`07-accessibility.md`

Accessibility

Purpose

Typography exists to communicate understanding.

If readers cannot comfortably perceive that understanding, the Typography System has failed regardless of how visually refined it appears.

Accessibility is therefore not an optional enhancement.

It is one of the primary architectural constraints of the Mosaic Typography System.

Typography should adapt to people.

People should never be expected to adapt to typography.

Philosophy

The Typography System follows one fundamental rule.

> ***Understanding must never depend upon perfect vision.***

Every typographic decision should preserve:

- hierarchy,
- rhythm,
- comprehension,
- orientation,

regardless of:

- visual ability,
- device,
- environment,
- operating system settings.

Accessibility strengthens typography.

It never compromises it.

Accessibility Before Density

When readability conflicts with information density:

Readability wins.

Always.

Poor.

`More Information`

↓

`Smaller Text`

Preferred.

`Comfortable Reading`

↓

`Progressive Disclosure`

The Composition Model already provides mechanisms for reducing density.

Typography should never become smaller simply because more information exists.

Hierarchy Preservation

Increasing text size should never flatten hierarchy.

Example.

Incorrect.

`Heading`

↓

`Body`

↓

`Both become similar.`

Preferred.

`Heading`

↓

`Scales`

↓

`Body`

↓

`Scales proportionally.`

Editorial relationships should remain immediately recognisable.

Accessibility should preserve understanding.

Not merely enlarge text.

Line Length

Larger text naturally changes reading measure.

Future implementations should compensate by adapting:

- line length,

- column width,
- paragraph spacing.

Comfortable reading should remain the primary objective.

Typography should never become a wall of enlarged text.

Line Spacing

Line spacing should increase alongside text size.

Reasons include:

- improved scanning,
- reduced visual crowding,
- lower cognitive effort.

The relationship between:

- font size,
- line height,
- paragraph spacing

should remain proportional.

Paragraph Rhythm

Accessibility should preserve editorial rhythm.

Large text.

↓

Larger paragraph separation.

Long-form reading should continue feeling natural.

The interface should never become visually compressed simply because text increased.

Colour Independence

Typography should remain understandable even if colour information is removed.

Examples.

Correct.

```
Heading
```

↓

```
Weight
```

↓

`Spacing`

↓

`Hierarchy`

Incorrect.

`Heading`

↓

`Blue Text`

Colour may reinforce hierarchy.

Typography should never depend upon it.

Contrast

Typography should maintain sufficient contrast under:

- Light Theme,
- Dark Theme,
- Runtime Atmosphere,
- High Contrast.

Atmosphere should automatically reduce whenever necessary to preserve readability.

Typography always possesses higher authority than atmosphere.

Runtime Atmosphere

Runtime Atmosphere should never reduce typographic clarity.

Examples.

Warm artwork.

↓

Subtle Material change.

↓

Typography unchanged.

Atmosphere belongs to materials.

Typography remains comparatively stable.

Reduced Motion

Typography should remain readable without motion.

Examples.

Transitions.

↓

Fade.

↓

No animated text movement required.

Understanding should never depend upon animation.

Motion may reinforce editorial rhythm.

It should never create it.

Screen Readers

Typography should preserve semantic structure.

Editorial hierarchy should map cleanly onto:

- headings,
- sections,
- paragraphs,
- captions.

Visual hierarchy and semantic hierarchy should remain aligned.

The interface should communicate the same structure whether read visually or through assistive technologies.

Variable Fonts

Future Variable Font implementations should support accessibility.

Possible runtime adaptations include:

- optical sizing,
- increased x-height,
- wider characters,
- reduced stroke contrast.

These adaptations should preserve the editorial voice of Mosaic while improving readability.

Reading Modes

Different reading contexts may require different accessibility strategies.

Watching.

↓

Minimal reading.

Reading.

↓

Maximum comfort.

Administration.

↓

Higher information density.

Accessibility should adapt according to the user's current activity rather than applying one global typographic profile.

Television

Television typography should account for:

- viewing distance,
- lower visual acuity,
- remote interaction.

Larger text alone is insufficient.

Future implementations should also consider:

- stronger hierarchy,
- simplified layouts,
- generous spacing.

Mobile

Mobile typography should account for:

- small displays,
- interruptions,
- varying lighting conditions.

The system should prioritise:

- short reading paths,
- comfortable tap targets,
- stable rhythm.

Reading should remain effortless despite environmental variability.

User Preferences

Future implementations should respect user preferences such as:

- preferred text size,
- increased contrast,
- bold text,
- operating system accessibility settings.

Platform preferences should integrate into the Typography Resolver rather than requiring application-specific logic.

Plugins

Extensions automatically inherit accessibility.

Plugins should never:

- override typography,
- introduce custom font scales,
- reduce readability.

Extensions contribute information.

The Typography System determines how that information becomes readable.

Good Examples

Reading

Larger text.

↓

Longer line spacing.

↓

Generous paragraphs.

↓

Comfortable rhythm.

The reading experience remains calm.

Playback

Overlay typography.

↓

High contrast.

↓

Clear hierarchy.

↓

Minimal atmospheric influence.

Interaction remains effortless.

Administration

Dense information.

↓

Responsive scaling.

↓

Stable editorial hierarchy.

↓

Clear navigation.

The interface remains functional without becoming overwhelming.

Anti-patterns

Tiny Metadata

Captions becoming unreadable.

Colour Hierarchy

Removing colour destroys editorial structure.

Flat Scaling

Every typographic role enlarged equally.

Hierarchy weakens.

Decorative Accessibility

Adding visual styling rather than improving comprehension.

Accessibility exists to improve understanding.

Accessibility Model

```
[mermaid]
flowchart TD

EditorialHierarchy
EditorialHierarchy --> Accessibility
Accessibility --> ResponsiveTypography
ResponsiveTypography --> RuntimeResolution
RuntimeResolution --> Presentation
```

Accessibility refines typography.

It never replaces editorial structure.

Relationship To Future Chapters

The next chapter defines **Runtime Typography Resolution**.

Accessibility explains:

> ***How typography adapts to different readers.***

Runtime Resolution explains:

> ***How those adaptations become concrete typography at runtime.***

Together they ensure the Companion remains readable everywhere.

Summary

Accessibility is one of the strongest architectural principles of the Mosaic Typography System.

Every user should experience the same:

- hierarchy,
- rhythm,
- understanding,
- editorial voice,

regardless of:

- ability,
- device,
- environment,
- display.

Typography succeeds when reading feels effortless.

Accessibility ensures that effortless experience belongs to everyone.

Review Status

Status

Draft

Next File

`08-runtime-resolution.md`

Runtime Typography Resolution

Purpose

The previous chapters established:

- Editorial Hierarchy
- Type Scale
- Reading Rhythm
- Hero Typography
- Responsive Typography
- Accessibility

This chapter defines how those concepts become concrete typography at runtime.

Runtime Typography Resolution is the mechanism through which abstract editorial intent becomes readable text on a specific device in a specific environment.

Applications should never determine:

- font size,
- line height,
- optical sizing,
- spacing,
- font variation,

independently.

Instead they request editorial intent.

The Typography System resolves everything else.

Definition

Within MDS, **Runtime Typography Resolution** is defined as:

> ***The deterministic process through which editorial typography is transformed into concrete runtime typography while preserving semantic hierarchy and reading comfort.***

Runtime Typography Resolution never changes editorial meaning.

It only determines how that meaning is expressed.

Why Resolution Exists

Without Runtime Resolution, every component would need to understand:

- screen size,

- viewing distance,
- accessibility,
- platform,
- font technology,
- user preferences.

Instead.

```text
Heading

↓

Runtime Resolver

↓

Rendered Typography
```

Components remain simple.

Typography remains consistent.

Resolution Pipeline

Every typographic role follows the same conceptual pipeline.

```text
Editorial Role

↓

Type Scale

↓

Responsive Rules

↓

Accessibility

↓

Runtime Context

↓

Platform Adaptation

↓

Resolved Typography
```

Each stage contributes one responsibility.

No stage duplicates another.

Resolution Inputs

Runtime Typography Resolution evaluates:

```
[text]
Editorial Role

↓

Current Context

↓

Device Class

↓

Viewing Distance

↓

Accessibility

↓

User Preferences

↓

Platform Capability
```

Typography adapts according to these inputs while preserving one editorial language.

Resolution Order

Typography should always resolve using the same conceptual order.

```
[text]
1.

Editorial Role

↓

2.

Type Scale

↓

3.

Reading Rhythm

↓

4.

Accessibility

↓

5.

Responsive Rules

↓
```

```
6.

Platform Adaptation

↓

7.

Rendered Typography
```

Meaning always precedes implementation.

Editorial Stability

One of the strongest guarantees within Mosaic is:

```
[text]
Heading

↓

Always Heading
```

The Heading may become:

- larger,
- smaller,
- wider,
- optically adjusted,

but it never stops being:

```
Heading
```

Applications should therefore consume editorial roles rather than implementation values.

Runtime Context

Typography may adapt according to the user's current activity.

Examples.

Watching.

↓

Less reading.

Reading.

↓

Comfort-first typography.

Administration.

↓

Higher density.

The same editorial role may therefore resolve differently while remaining conceptually identical.

Viewing Distance

Viewing distance is considered a runtime input.

Television.

↓

Greater physical size.

Phone.

↓

Smaller physical size.

Desktop.

↓

Balanced implementation.

The user should perceive equivalent readability regardless of hardware.

Accessibility Resolution

Accessibility possesses higher authority than responsive behaviour.

Examples.

Large Text.

↓

Larger scale.

↓

Greater line spacing.

↓

Paragraph rhythm preserved.

Reduced Vision.

↓

Increased optical size.

↓

Higher readability.

Editorial rhythm should remain recognisable after accessibility adaptation.

Variable Font Resolution

Future implementations may resolve:

- weight,
- width,
- optical size,
- grade,

using variable font technology.

Applications should never manipulate these axes directly.

They simply request:

```
[text]
Heading
```

The Typography Resolver determines the appropriate implementation.

Runtime Caching

Resolved typography should be cacheable.

Typical invalidation events include:

- accessibility changes,
- device changes,
- orientation changes,
- platform font changes.

Ordinary interaction should not repeatedly resolve typography.

Reading should remain visually stable.

Incremental Resolution

Typography updates should occur incrementally.

Preferred.

```
[text]
Accessibility Changes

↓

Body

↓

Supporting

↓

Caption
```

Avoid.

```
[text]
Accessibility Changes

↓

Entire Typography System Rebuilt
```

Readers should perceive continuity.

Not replacement.

Platform Adaptation

Different rendering technologies possess different capabilities.

Examples.

Web.

↓

Variable font support.

Flutter.

↓

Engine-specific metrics.

Television.

↓

Distance optimisation.

The Typography Resolver should compensate for implementation differences while preserving editorial consistency.

Material Awareness

Typography should respect surrounding Materials.

Hero Material.

↓

Slightly greater spacing.

Overlay Material.

↓

Higher contrast.

Canvas.

↓

Editorial rhythm.

Materials influence implementation.

They never redefine editorial hierarchy.

Runtime Atmosphere

Typography should participate minimally in Runtime Atmosphere.

Atmosphere may subtly influence:

- luminance,
- contrast,

It should never influence:

- hierarchy,
- weight,
- reading rhythm.

Words remain the most stable visual element within the interface.

Plugins

Extensions never resolve typography.

Plugins contribute:

- information,
- descriptions,
- metadata.

The Typography Resolver determines:

- hierarchy,
- spacing,
- scaling,
- accessibility.

Every extension therefore automatically inherits future typographic improvements.

Good Examples

Hero

Hero Title.

↓

Responsive Scale.

↓

Accessibility.

↓

Resolved Typography.

↓

Presentation.

Reading

Body.

↓

Comfort Profile.

↓

Reading Rhythm.

↓

Rendered Paragraphs.

The experience remains editorial rather than technical.

Administration

Supporting Text.

↓

Higher Density Profile.

↓

Accessible Scaling.

↓

Presentation.

Information remains efficient without abandoning Mosaic's typographic voice.

Anti-patterns

Component Typography

Components selecting font sizes independently.

Platform Typography

Each client inventing independent typography behaviour.

Runtime Identity

Runtime changing Heading into Body.

Editorial meaning has leaked into implementation.

Atmosphere Typography

Artwork directly influencing font weights or scale.

Atmosphere belongs to materials.

Not language.

Runtime Typography Model

```
[mermaid]
flowchart TD

EditorialRole
EditorialRole --> TypeScale
TypeScale --> ReadingRhythm
ReadingRhythm --> Accessibility
Accessibility --> ResponsiveRules
ResponsiveRules --> Platform
Platform --> ResolvedTypography
ResolvedTypography --> Presentation
```

Applications request editorial intent.

The Typography System resolves implementation.

Relationship To Future Chapters

The next chapter defines **Platform Typography**.

Runtime Resolution explains:

> ***How typography becomes concrete at runtime.***

Platform Typography explains:

> ***How different rendering technologies implement that typography while preserving one editorial language.***

Together they complete the runtime architecture of the Typography System.

Summary

Runtime Typography Resolution transforms editorial intent into readable typography.

It preserves:

- hierarchy,

- rhythm,
- accessibility,
- responsiveness,
- consistency,

while hiding implementation complexity from applications.

Components should never ask:

> ***"What font size should I use?"***

They should simply request:

> ***Heading***

The Typography System does everything else.

Review Status

Status

Draft

Next File

`09-platform-typography.md`

Platform Typography

Purpose

The Typography System defines one editorial language.

Mosaic clients implement that language across many rendering technologies.

Platform Typography ensures that:

- Web
- Flutter
- SwiftUI
- Jetpack Compose
- Desktop
- Television
- Future clients

all communicate the same editorial experience despite differing implementation capabilities.

Platform implementation should never redefine editorial meaning.

It should simply express it.

Definition

Within MDS, **Platform Typography** is defined as:

> ***The platform-specific implementation of the Mosaic Typography System while preserving one consistent editorial language.***

Platform Typography concerns implementation.

It does not concern design philosophy.

Philosophy

Every platform possesses different capabilities.

Examples include:

- font rendering
- anti-aliasing
- shaping engines
- variable font support
- hinting

- sub-pixel rendering

These differences are implementation concerns.

Readers should experience one consistent Companion regardless of platform.

Platform Independence

Editorial meaning should remain platform independent.

Conceptually.

```
[text]
Heading

↓

Web

Heading

↓

Flutter

Heading

↓

SwiftUI

Heading

↓

Compose
```

Only rendering changes.

Meaning remains identical.

Platform Responsibilities

Each platform implementation is responsible for:

- font loading
- shaping
- rendering
- fallback fonts
- locale support
- glyph rasterisation

Platforms are **not** responsible for:

- editorial hierarchy
- type scale

- reading rhythm
- semantic typography

Those responsibilities belong to the Typography System.

Typography Resolver

Every platform should consume the output of the Typography Resolver.

Conceptually.

```
[text]
Editorial Role

↓

Typography Resolver

↓

Platform Typography

↓

Rendering
```

Platforms should never reinterpret editorial roles.

Rendering Quality

Platform implementations should prioritise:

- readability
- consistency
- stability

before:

- rendering tricks
- custom effects
- decorative styling

Typography exists to communicate understanding.

Rendering quality should strengthen that communication.

Font Metrics

Different rendering engines interpret font metrics differently.

Examples include:

- ascender height
- descender height

- x-height
- line height
- baseline

Future platform adapters should compensate for these differences.

Users should perceive one typographic rhythm regardless of implementation.

Font Fallback

Platform Typography should define predictable fallback behaviour.

Conceptually.

```text
Primary Typeface
```

↓

```
Platform Fallback
```

↓

```
System Typeface
```

Fallbacks should preserve:

- hierarchy
- rhythm
- readability

Editorial identity should remain recognisable even when the preferred font is unavailable.

Variable Fonts

Platforms supporting variable fonts should use them.

Platforms without variable font support should approximate the same editorial result using discrete font weights.

The editorial role remains identical.

Only implementation differs.

International Typography

Platform implementations should support multilingual typography.

Examples include:

- Latin
- Cyrillic
- Greek

- Arabic
- Hebrew
- Devanagari
- Japanese
- Korean
- Chinese

Editorial hierarchy should remain consistent regardless of writing system.

The Typography System should feel culturally adaptable rather than culturally fixed.

Bidirectional Text

Future implementations should support bidirectional text.

Editorial hierarchy should remain identical for:

- left-to-right
- right-to-left
- mixed-direction documents

The reading model adapts.

The editorial philosophy does not.

Responsive Rendering

Platform typography should adapt to:

- display density
- operating system scaling
- viewing distance
- accessibility

The implementation may differ.

The perceived reading comfort should remain stable.

Television

Television typography should optimise for:

- distance
- larger viewing angles
- remote interaction

Rendering should favour clarity over compactness.

Long reading passages should remain comfortable despite increased viewing distance.

Mobile

Mobile typography should optimise for:

- close viewing
- interruption
- one-handed interaction

Editorial rhythm should remain intact despite constrained space.

Desktop

Desktop typography should take advantage of:

- additional space
- longer reading sessions
- precise rendering

Extra space should improve rhythm.

It should not increase visual complexity.

Performance

Platform implementations should:

- cache glyphs
- reuse layouts
- minimise recomposition
- preserve rendering stability

Typography should remain visually stable throughout interaction.

Rendering should never distract from reading.

Plugins

Extensions should never provide platform-specific typography.

Plugins contribute:

- information
- language
- editorial content

The platform determines typography.

This guarantees one editorial voice throughout the ecosystem.

Good Examples

Web

Variable fonts.

↓

Editorial hierarchy preserved.

↓

Reading rhythm preserved.

Flutter

Platform text engine.

↓

Typography Resolver.

↓

Identical editorial behaviour.

Television

Larger physical typography.

↓

Greater spacing.

↓

Equivalent reading comfort.

Anti-patterns

Platform Fonts

Each client invents different editorial hierarchy.

Platform Styling

Platform-specific decorative typography.

Manual Scaling

Application code selecting typography independently.

Decorative Rendering

Rendering effects reducing readability.

Platform Typography Model

```
[mermaid]
flowchart TD

EditorialRole
EditorialRole --> TypographyResolver
TypographyResolver --> PlatformAdapter
PlatformAdapter --> Web
PlatformAdapter --> Flutter
PlatformAdapter --> SwiftUI
PlatformAdapter --> Compose
PlatformAdapter --> Television
Web --> Presentation
Flutter --> Presentation
SwiftUI --> Presentation
Compose --> Presentation
Television --> Presentation
```

One editorial language.

Many platform implementations.

Relationship To Future Chapter

The next chapter defines **Variable Fonts**.

Platform Typography explains:

> ***How platforms implement typography.***

Variable Fonts explain:

> ***How typography becomes more adaptive while preserving editorial consistency.***

Together they complete the implementation architecture of the Typography System.

Summary

Platform Typography exists to ensure that Mosaic speaks with one voice everywhere.

Different rendering engines may implement typography differently.

Readers should never perceive those differences.

They should simply feel that the Companion remains:

- calm,

- confident,
- familiar,

regardless of the device in their hands.

Review Status

Status

Draft

Next File

`10-variable-fonts.md`

Variable Fonts

Purpose

Variable Fonts provide one of the most powerful implementation capabilities available to the Mosaic Typography System.

However...

Within Mosaic they are intentionally treated as an implementation technique rather than a design feature.

Contributors should never think:

> *"How can we use variable fonts?"*

Instead they should ask:

> ***"How can variable typography better communicate understanding?"***

Variable Fonts exist to support the editorial language.

They never define it.

Definition

Within MDS, **Variable Fonts** are defined as:

> ***Typography capable of continuously adapting multiple visual characteristics while preserving one stable editorial language.***

Variable Fonts provide flexibility.

Editorial hierarchy provides meaning.

Philosophy

Typography should adapt to readers.

Not force readers to adapt to typography.

Variable Fonts allow Mosaic to subtly improve:

- readability,
- hierarchy,
- rhythm,
- accessibility,
- device adaptation,

without introducing additional font families or breaking editorial consistency.

The adaptation should feel invisible.

Why Variable Fonts Exist

Traditional typography requires discrete font files.

Examples.

```
Regular

Medium

SemiBold

Bold
```

Variable Fonts instead expose continuous typographic behaviour.

Conceptually.

```
[text]
Weight

↓

Continuous

Width

↓

Continuous

Optical Size

↓

Continuous

Grade

↓

Continuous
```

The Typography Resolver determines appropriate values.

Applications remain unaware of these details.

Supported Axes

The Typography System currently recognises the following conceptual variable axes.

```
Weight

↓

Width

↓

Optical Size

↓

Grade
```

Future axes may be supported provided they reinforce editorial understanding rather than visual novelty.

Weight

Purpose.

Communicate hierarchy.

Examples.

```
Heading
```

↓

```
Higher Weight
```

```
Body
```

↓

```
Moderate Weight
```

Weight should communicate confidence rather than heaviness.

Avoid dramatic weight differences unless editorially justified.

Width

Purpose.

Optimise reading comfort.

Examples.

Phone.

↓

Slightly wider characters.

Television.

↓

Slightly narrower measure.

Width should improve reading.

Not create stylistic variation.

Optical Size

Purpose.

Optimise typography for its rendered size.

Smaller text may require:

- larger counters,
- increased spacing,

- adjusted stroke contrast.

Larger typography may become more refined.

Applications should never manipulate Optical Size directly.

The Typography Resolver owns these decisions.

Grade

Purpose.

Improve readability without significantly affecting layout.

Examples.

High contrast.

↓

Slightly increased grade.

Low brightness.

↓

Slightly heavier grade.

Grade differs from Weight.

Weight influences hierarchy.

Grade primarily influences readability.

This distinction becomes increasingly valuable for accessibility.

Runtime Adaptation

Variable Fonts should adapt according to:

- viewing distance,
- accessibility,
- platform,
- display density.

They should **not** adapt according to:

- artwork,
- runtime atmosphere,
- media genre.

Typography remains editorially stable.

Materials carry environmental adaptation.

Editorial Stability

One of the strongest guarantees within Mosaic is:

```
[text]
Heading
```

↓

```
Always Heading
```

Variable Font adjustments should never change editorial meaning.

They simply optimise implementation.

Users should perceive one consistent editorial voice.

Responsive Behaviour

Future implementations may use Variable Fonts to preserve reading rhythm.

Example.

Desktop.

↓

Default optical size.

Phone.

↓

Closer viewing distance.

↓

Adjusted optical size.

Television.

↓

Greater distance.

↓

Adjusted weight.

The editorial hierarchy remains identical.

Only implementation adapts.

Accessibility

Variable Fonts provide significant accessibility advantages.

Examples include:

- larger x-height,

- improved letter distinction,
- stronger grade,
- refined optical sizing.

Accessibility adaptations should remain subtle.

Readers should perceive increased comfort rather than dramatically different typography.

Long-Form Reading

Books.

Reviews.

Descriptions.

Editorial content.

These experiences should receive the greatest benefit from Variable Fonts.

Future Typography Resolvers may optimise:

- paragraph rhythm,
- optical size,
- line spacing,
- grade,

according to reading duration.

Reading should become progressively more comfortable rather than mechanically consistent.

Hero Typography

Hero Typography should use Variable Fonts conservatively.

Examples.

Slight optical refinement.

Subtle weight adjustment.

Improved spacing.

Avoid dramatic stylistic variation.

The Hero should communicate calm confidence rather than typographic experimentation.

Cross-Platform Behaviour

Platforms supporting Variable Fonts should use them.

Platforms without support should approximate equivalent behaviour using traditional font weights.

The editorial language should remain identical.

Readers should never perceive which implementation strategy is being used.

Runtime Resolution

Variable Fonts should participate within Runtime Typography Resolution.

Conceptually.

```text
Editorial Role

↓

Typography Resolver

↓

Variable Font Axes

↓

Resolved Typography

↓

Presentation
```

Applications request editorial intent.

The resolver determines font behaviour.

Performance

Future implementations should:

- cache resolved typography,
- minimise axis recomputation,
- reuse layouts,
- preserve rendering stability.

Variable Fonts should improve flexibility without introducing perceptible performance costs.

Plugins

Extensions should never manipulate Variable Font axes.

Plugins contribute:

- language,
- information,
- editorial content.

The Typography Resolver determines:

- weight,

- width,
- optical size,
- grade.

Every extension therefore inherits future typography improvements automatically.

Good Examples

Phone

Heading.

↓

Slight optical adjustment.

↓

Improved readability.

The reader notices comfort rather than typography.

Television

Body.

↓

Greater viewing distance.

↓

Adjusted weight.

↓

Equivalent reading comfort.

Reading

Long-form content.

↓

Runtime reading profile.

↓

Optimised optical size.

↓

Comfortable editorial rhythm.

Anti-patterns

Decorative Variation

Constant weight changes purely for visual interest.

Artwork Typography

Artwork influencing Variable Font axes.

Platform Typography

Each client independently selecting variable axes.

Runtime Styling

Variable Fonts responding dramatically to atmosphere.

Typography should remain stable.

Variable Font Model

```
[mermaid]
flowchart TD

EditorialRole
EditorialRole --> TypographyResolver
TypographyResolver --> VariableFontAxes
VariableFontAxes --> Weight
VariableFontAxes --> Width
VariableFontAxes --> OpticalSize
VariableFontAxes --> Grade
Weight --> Presentation
Width --> Presentation
OpticalSize --> Presentation
Grade --> Presentation
```

Variable Fonts refine typography.

They never redefine editorial meaning.

Relationship To Future Chapters

The next chapter defines **Typography Governance**.

Variable Fonts explain:

> ***How typography becomes more adaptive.***

Governance explains:

> ***How that adaptation remains architecturally consistent over the lifetime of Mosaic.***

Together they ensure flexibility without sacrificing editorial identity.

Summary

Variable Fonts allow Mosaic to quietly adapt typography to:

- readers,
- devices,
- accessibility,
- environments.

The adaptation should feel invisible.

Readers should simply experience typography that always feels comfortable, calm and unmistakably Mosaic.

The technology exists to serve the editorial language.

Never to replace it.

Review Status

Status

Draft

Next File

`11-governance.md`

Typography System Governance

Purpose

Typography is one of the most recognisable characteristics of a product.

Users may never consciously remember:

- font families,
- point sizes,
- variable font axes.

They will remember:

- whether reading felt effortless,
- whether the interface sounded calm,
- whether the product felt trustworthy.

Typography therefore becomes part of the long-term identity of Mosaic.

This chapter defines how the Typography System should evolve while preserving that identity.

Governance Philosophy

Typography should evolve technologically.

Its editorial voice should remain remarkably stable.

The objective is not preserving fonts.

It is preserving:

- editorial rhythm,
- hierarchy,
- readability,
- companionship.

Rendering technologies may change.

Typography should continue sounding like the same Companion.

Typography Is Architecture

Within Mosaic, typography is treated as architectural language rather than visual styling.

Changing:

```
Heading
```

affects:

- hierarchy,
- composition,
- reading rhythm,
- accessibility,
- every supported client.

Typography changes should therefore receive architectural review.

Not merely visual approval.

Stable Responsibilities

The following concepts should remain highly stable.

- Editorial Hierarchy
- Reading Rhythm
- Hero Typography
- Typographic Voice
- Editorial Roles
- Reading Philosophy

These concepts define the public typographic language of Mosaic.

Evolvable Responsibilities

The following may evolve more frequently.

- font family
- variable font technology
- rendering engines
- optical sizing algorithms
- runtime scaling
- platform-specific typography

Implementation may evolve.

Editorial meaning should remain stable.

Typography Ownership

Typography responsibilities are intentionally separated.

Layer	Owner
Typography Philosophy	Design Systems

Layer	Owner
Editorial Hierarchy	Design Systems
Runtime Typography Resolver	Runtime Platform
Platform Rendering	Client Platform
Font Implementation	Platform Teams

Ownership preserves consistency while allowing implementation to improve independently.

Introducing New Typography

Before introducing a new typographic role ask:

Question One

Does an existing editorial role already communicate this meaning?

Question Two

Is this genuinely a new reading behaviour...

or merely another visual treatment?

Question Three

Could spacing, hierarchy or composition solve this instead?

Question Four

Will this role remain meaningful after a redesign?

Question Five

Would another contributor naturally discover this role?

If uncertainty remains...

Refinement should continue before implementation.

Editorial Drift

Editorial Drift occurs when:

- headings begin behaving differently,
- platforms invent new hierarchy,
- typography becomes decorative,

- marketing styles leak into runtime,
- multiple editorial voices emerge.

Editorial Drift weakens the Companion.

Users may not consciously notice it.

They simply begin feeling that the product has become inconsistent.

Typography Debt

Typography Debt accumulates through:

- duplicate editorial roles,
- inconsistent spacing,
- decorative exceptions,
- platform-specific hierarchy,
- component-owned typography.

Typography Debt should be treated as architectural debt.

Reducing it often improves readability more than introducing new features.

Runtime Governance

Runtime Typography Resolution may evolve continuously.

Examples include:

- better variable font support,
- improved optical sizing,
- smarter accessibility,
- adaptive reading profiles.

These improvements should require no changes to:

- Components,
- Composition,
- Editorial Hierarchy.

Applications should remain unaware of implementation improvements.

Accessibility Governance

Accessibility always possesses higher authority than typography aesthetics.

If typography choices reduce:

- readability,

- rhythm,
- comprehension,

they should be reconsidered regardless of visual appeal.

Typography exists to communicate.

Not impress.

Platform Governance

Every platform should communicate the same editorial language.

Desktop.

↓

Editorial.

Television.

↓

Editorial.

Phone.

↓

Editorial.

Implementation differences are expected.

Editorial differences are not.

Plugin Governance

Extensions should never introduce:

- custom typography scales,
- independent hierarchy,
- alternative editorial voices,
- decorative fonts.

Plugins inherit:

- Typography Roles,
- Reading Rhythm,
- Accessibility,
- Runtime Resolution.

This guarantees one coherent editorial language throughout the ecosystem.

Review Questions

Every typography proposal should answer:

- Does this strengthen understanding?
- Does this preserve editorial rhythm?
- Does this improve reading comfort?
- Does this reinforce the Companion?
- Would users still recognise Mosaic after this change?
- Is this solving a reading problem or adding visual novelty?

If the proposal exists primarily because it "looks better", it should be reconsidered.

Validation

Future tooling should automatically validate:

- editorial hierarchy
- typography token usage
- accessibility compliance
- runtime consistency
- responsive behaviour
- platform parity

Validation should reinforce architectural review.

It should never replace thoughtful editorial judgement.

Governance Workflow

```
[mermaid]
flowchart TD

Proposal
Proposal --> EditorialReview
EditorialReview --> ExistingRole["ExistingRole?"]
ExistingRole["ExistingRole?"] -->|Yes| Reuse
ExistingRole["ExistingRole?"] -->|No| ArchitectureReview
ArchitectureReview
ArchitectureReview --> AccessibilityReview
AccessibilityReview --> RuntimeReview
RuntimeReview --> Approval
Approval --> Implementation
```

Typography should evolve through refinement rather than expansion.

Success Criteria

The Typography System succeeds when:

- users instinctively know what to read first,
- reading feels calm,
- editorial rhythm remains consistent,
- accessibility remains effortless,
- contributors naturally reuse existing editorial roles,
- every Mosaic client sounds like the same Companion.

Typography should become invisible.

Only understanding should remain.

Architectural Decisions

ADR	Decision
ADR-120	Typography is treated as editorial architecture rather than visual styling.
ADR-121	Editorial hierarchy is a stable public design contract.
ADR-122	Runtime Typography Resolution owns implementation while preserving editorial meaning.
ADR-123	Accessibility always has higher authority than typographic aesthetics.
ADR-124	Extensions inherit the Typography System rather than extending it.

Review Status

Status

Draft

Next File

`12-adrs.md`

Architectural Decision Records

Purpose

The Architectural Decision Records (ADRs) contained within MDS-004 preserve the architectural reasoning behind the Mosaic Typography System.

Where previous specifications established:

- Design Tokens
- Colour
- Materials

MDS-004 establishes the editorial voice through which the Mosaic Companion communicates.

These ADRs explain why Mosaic deliberately treats typography as language rather than decoration.

Future contributors should understand these decisions before proposing changes to typography behaviour or hierarchy.

ADR Format

Every Mosaic ADR follows the standard structure.

```
[text]
ADR Number

Status

Context

Decision

Consequences

Alternatives Considered

Related Specifications
```

Each ADR records one significant architectural decision.

ADR-125

Title

Treat Typography As Editorial Architecture

Status

Accepted

Context

Many interface systems optimise typography for information density.

Founder workshops consistently described Mosaic as a Companion rather than a dashboard.

Decision

Typography becomes editorial architecture rather than interface styling.

Consequences

Reading becomes calmer, more natural and more immersive across every Mosaic client.

ADR-126

Title

Editorial Hierarchy Drives Typography

Status

Accepted

Context

Allowing typography to determine hierarchy creates inconsistent experiences.

Decision

Composition defines hierarchy.

Typography expresses hierarchy.

Consequences

Editorial consistency is preserved across every device and future redesign.

ADR-127

Title

Separate Editorial Roles From Font Sizes

Status

Accepted

Context

Pixel-based typography tightly couples application code to implementation.

Decision

Applications consume editorial roles such as:

- Display
- Heading

- Section
- Body
- Supporting
- Caption

Runtime determines physical implementation.

Consequences

Typography becomes platform independent and future proof.

ADR-128

Title

Runtime Typography Owns Adaptation

Status

Accepted

Context

Accessibility, viewing distance and device capabilities require adaptive typography.

Decision

The Runtime Typography Resolver becomes solely responsible for runtime adaptation.

Applications remain unaware of implementation details.

Consequences

Typography adapts continuously while preserving one editorial language.

ADR-129

Title

Reading Comfort Has Higher Priority Than Density

Status

Accepted

Context

Many media applications sacrifice reading comfort to display additional information.

Decision

Reading comfort always takes precedence over interface density.

Composition should reduce information before typography becomes uncomfortable.

Consequences

Long-form reading becomes significantly more enjoyable.

ADR-130

Title

Hero Typography Introduces Rather Than Advertises

Status

Accepted

Context

Traditional Hero typography frequently behaves like marketing.

Founder workshops consistently favoured calm editorial presentation.

Decision

Hero Typography communicates confidence through restraint rather than spectacle.

Consequences

Entertainment remains emotionally dominant while typography quietly establishes orientation.

ADR-131

Title

Accessibility Overrides Typography Aesthetics

Status

Accepted

Context

Decorative typography frequently reduces readability.

Decision

Accessibility possesses higher authority than typographic styling.

Consequences

Typography remains understandable regardless of:

- theme,
- device,

- accessibility settings,
- runtime adaptation.

ADR-132

Title

Variable Fonts Are An Implementation Detail

Status

Accepted

Context

Variable Fonts provide significant flexibility but should not influence editorial architecture.

Decision

Variable Fonts remain entirely behind the Runtime Typography Resolver.

Applications consume editorial roles.

Consequences

Future typography technologies may evolve without affecting application code.

ADR-133

Title

Extensions Inherit Typography

Status

Accepted

Context

Allowing extensions to introduce independent typographic systems fragments product identity.

Decision

Extensions contribute editorial content.

The platform owns typography.

Consequences

Community extensions inherit future typography improvements automatically.

ADR Relationships

```
[mermaid]
flowchart TD

ADR125["Editorial Architecture"]

ADR126["Hierarchy"]

ADR127["Editorial Roles"]

ADR128["Runtime"]

ADR129["Reading Comfort"]

ADR130["Hero"]

ADR131["Accessibility"]

ADR132["Variable Fonts"]

ADR133["Extensions"]

ADR125 --> ADR126
ADR126 --> ADR127
ADR127 --> ADR128
ADR128 --> ADR132
ADR129 --> ADR127
ADR130 --> ADR126
ADR131 --> ADR128
ADR128 --> ADR133
```

Together these decisions establish typography as the editorial voice of the Mosaic Companion.

Future ADRs

Future Typography ADRs are expected to formalise:

- AI-assisted Reading Profiles
- Adaptive Reading Tempo
- Cross-Language Editorial Behaviour
- Dynamic Optical Scaling
- Immersive Reading Mode
- Television Typography Profiles
- Accessibility Reading Personas
- Editorial Voice Localisation

These intentionally remain outside the scope of MDS-004 Version 0.1.

ADR Governance

Typography ADRs should change only when:

- editorial research identifies deficiencies,

- accessibility research requires refinement,
- runtime typography architecture evolves,
- the Design Language itself changes.

Rendering technology alone should never justify architectural changes.

Typography should remain recognisably Mosaic regardless of implementation.

Summary

The ADRs contained within MDS-004 define the editorial identity of Mosaic.

Typography is not treated as decoration.

It is treated as language.

Every implementation should therefore preserve:

- calmness,
- hierarchy,
- reading rhythm,
- companionship,

while allowing rendering technology to evolve independently.

Review Status

Status

Draft

Next File

`13-contributor-guidance.md`

Contributor Guidance

Purpose

Typography is one of the few systems every Mosaic contributor will influence, whether intentionally or not.

Designers choose hierarchy.

Engineers implement typography.

Extension authors contribute language.

Writers define content.

The purpose of this guidance is to ensure every contributor strengthens one coherent editorial voice.

The user should always feel they are interacting with the same Companion.

Think In Editorial Roles

Never begin with:

> *"What font size should this use?"*

Instead ask:

> ***"What editorial role does this text perform?"***

Good.

```
Heading
```

Poor.

```
24px
```

Editorial meaning should always precede implementation.

Write Before Styling

Typography exists to communicate language.

Good process.

```
[text]
Meaning

↓

Language

↓

Editorial Role

↓

Typography
```

↓

`Presentation`

Poor process.

```
[text]
Typography
```

↓

`Visual Style`

↓

`Write Content`

The words should determine the typography.

Never the reverse.

One Editorial Voice

Every part of Mosaic should sound as though it comes from the same Companion.

Examples include:

- onboarding,
- playback,
- search,
- administration,
- settings,
- extensions.

Tone may adapt slightly.

Voice should remain consistent.

Users should never feel they have entered another application.

Respect Reading Rhythm

Before introducing additional typography ask:

> *Does this improve reading...*

or simply add another label?

Typography should reduce effort.

Not increase it.

Whenever possible:

Remove words before reducing typography.

Prefer Hierarchy Over Decoration

If emphasis is required:

Prefer:

- Composition
- spacing
- hierarchy
- editorial structure

before introducing:

- heavier weights,
- larger sizes,
- brighter colours.

Typography should communicate confidence through restraint.

Components Consume Roles

Components should never request:

```
18px
```

```
Bold
```

Instead request:

```
Heading
```

or

```
Supporting
```

The Typography Resolver owns physical implementation.

Applications should remain implementation independent.

Let Materials Speak

Typography exists inside Materials.

Do not attempt to compete with them.

Hero Material already communicates importance.

Typography should reinforce that importance.

Not duplicate it.

Respect Accessibility

Whenever typography choices conflict with readability:

Readability wins.

Examples.

Poor.

`Beautiful`

↓

`Unreadable`

Preferred.

`Readable`

↓

`Beautiful`

Typography exists to communicate.

Everything else is secondary.

Responsive Thinking

Avoid asking:

> *"How does this shrink?"*

Instead ask:

> *"How should this read on another device?"*

The editorial role should remain identical.

Only the implementation changes.

Variable Fonts

Contributors should never manipulate:

- weight,
- optical size,
- width,
- grade

directly.

Applications request editorial roles.

The Typography Resolver determines appropriate variable font behaviour.

Extensions

Extensions should contribute:

- titles,
- descriptions,
- metadata,
- editorial content.

They should never define:

- font families,
- scales,
- hierarchy,
- spacing.

The platform owns typography.

Extensions inherit it.

Language

Typography cannot compensate for poor writing.

Before adjusting typography ask:

> *Can the text itself become clearer?*

Simple language usually improves understanding more than visual refinement.

Good typography amplifies good writing.

It rarely rescues poor writing.

Common Mistakes

Avoid the following.

Pixel Thinking

Choosing typography using measurements rather than editorial roles.

Marketing Language

Every heading sounding promotional.

Decorative Hierarchy

Typography competing with entertainment.

Platform Typography

Each client inventing independent editorial behaviour.

Component Fonts

Components selecting their own typography.

Dense Interfaces

Reducing text size instead of improving Composition.

Typography Review Questions

Before implementing typography ask:

- What editorial role does this perform?
- Does it strengthen reading rhythm?
- Does it reinforce Composition?
- Will it remain understandable without colour?
- Would this still feel like Mosaic after a redesign?
- Does it sound like the Companion?

If uncertainty remains...

Return to the editorial hierarchy before implementation.

Typography Checklist

Every typography implementation should satisfy the following.

- [ ] Editorial role is clearly defined.
- [ ] Reading rhythm is preserved.
- [ ] Accessibility is maintained.
- [ ] Responsive behaviour is supported.
- [ ] Components consume editorial roles.
- [ ] Runtime Typography Resolver owns implementation.
- [ ] Platform independence is preserved.
- [ ] Typography supports rather than competes with entertainment.

Final Guidance

The Typography System should eventually disappear from conscious thought.

Contributors should stop asking:

> *"What font should this use?"*

and instinctively begin asking:

> ***"How should the Companion say this?"***

When every contributor naturally thinks in editorial language rather than typography, the system has achieved its purpose.

Readers will simply experience calm, confident understanding.

That is the ultimate objective of the Mosaic Typography System.

Review Status

Status

Draft

Next File

`glossary.md`

Glossary

Purpose

This glossary defines the terminology introduced by **MDS-004 — Typography System**.

Unlike previous glossaries, these definitions describe the architectural language of typography rather than implementation details.

These definitions should remain stable regardless of:

- rendering technology,
- font family,
- platform,
- device.

Future specifications should reuse these terms consistently.

A

Accessibility Typography

Typography resolved according to user accessibility requirements while preserving editorial hierarchy and reading rhythm.

Accessibility Typography strengthens readability without introducing a different editorial language.

B

Body

The primary editorial role for sustained reading.

Body typography is intended for:

- descriptions,
- reviews,
- biographies,
- long-form editorial content.

Body represents the default reading voice of the Companion.

C

Caption

The quietest editorial role.

Captions communicate:

- technical metadata,
- timestamps,
- diagnostics,
- secondary context.

Caption should remain readable while avoiding unnecessary visual emphasis.

D

Display

The highest editorial role.

Display typography is reserved for rare moments such as:

- Hero titles,
- onboarding,
- major transitions.

Display should communicate calm confidence rather than spectacle.

E

Editorial Hierarchy

The ordered organisation of written language according to conceptual importance.

Editorial Hierarchy is derived from Composition.

Typography expresses it.

Editorial Role

A semantic typography role independent from physical implementation.

Examples include:

- Display
- Heading
- Section

- Body
- Supporting
- Caption

Applications should consume editorial roles rather than font sizes.

H

Heading

The editorial role responsible for introducing major concepts.

Examples include:

- film titles,
- book titles,
- artist names,
- collection titles.

Headings establish orientation rather than decoration.

Hero Typography

The editorial expression of the current Hero.

Hero Typography introduces the user's current Focus while preserving the emotional dominance of entertainment artwork.

O

Optical Size

A variable font axis used to improve readability at different rendered sizes.

Optical Size is determined by the Typography Resolver.

Applications should never manipulate it directly.

P

Paragraph Rhythm

The pacing created by:

- paragraph spacing,
- line spacing,
- editorial hierarchy.

Paragraph Rhythm contributes significantly to long-form reading comfort.

Platform Typography

The platform-specific implementation of the Mosaic Typography System.

Platform Typography preserves editorial meaning while adapting rendering for:

- Web,
- Flutter,
- SwiftUI,
- Compose,
- Television,
- future clients.

R

Reading Rhythm

The deliberate pacing of typography and spacing that guides readers naturally through a Composition.

Reading Rhythm transforms information into a comfortable reading experience.

Responsive Typography

Typography that adapts to different viewing environments while preserving editorial hierarchy.

Responsive Typography changes implementation.

It does not change meaning.

Runtime Typography Resolver

The runtime subsystem responsible for converting editorial roles into concrete typographic implementation.

Applications consume resolved typography rather than implementing typography themselves.

S

Section

The editorial role responsible for organising major conceptual groups.

Examples include:

- Continue Watching,
- Cast,

- Chapters,
- Related Works.

Section typography structures understanding.

Supporting

The editorial role used for secondary information.

Examples include:

- runtime,
- author,
- release year,
- language.

Supporting typography remains readable while remaining visually quieter than Body.

T

Type Scale

The ordered system of editorial typography roles.

The Type Scale expresses hierarchy.

It does not define hierarchy.

Typography Resolver

The conceptual runtime process that transforms editorial roles into resolved typography according to:

- accessibility,
- responsive rules,
- platform,
- viewing environment.

V

Variable Font

A font capable of continuously adapting properties such as:

- weight,
- width,
- optical size,

- grade.

Within Mosaic, Variable Fonts remain an implementation technique.

Editorial meaning remains unchanged.

W

Weight

The perceived visual strength of typography.

Weight communicates hierarchy.

It should never become decorative.

Within Mosaic, Weight is resolved by the Typography Resolver rather than selected manually.

Cross References

Specification	Primary Concepts
MDL-001 Vision	Companion, Immersion
MDL-002 Principles	Calm Interfaces
MDL-003 Mental Model	World, Focus
MDL-004 Interaction Model	Reading Behaviour
MDL-005 Composition Model	Hero, Hierarchy
MDS-001 Design Token Architecture	Semantic Roles
MDS-003 Material System	Hero Material, Canvas

Terminology Rules

Future contributors should:

- describe editorial roles before font sizes,
- describe reading before rendering,
- distinguish hierarchy from typography,
- distinguish editorial language from implementation,
- avoid platform-specific terminology inside architectural specifications.

Typography terminology should remain independent from rendering technology.

Review Status

Status

Draft

Next File

`references.md`

References

Purpose

This document records the architectural influences and conceptual foundations that informed **MDS-004 — Typography System**.

Unlike implementation documentation, these references explain *why* Mosaic speaks the way it does rather than prescribing specific font technologies or rendering techniques.

The Typography System intentionally combines ideas from:

- editorial design,
- book typography,
- information architecture,
- human perception,
- accessibility,
- responsive reading,

into one coherent editorial language centred around companionship rather than software.

Reading Order

Contributors should approach references in the following order.

1. MDL Specifications 2. Design Token Architecture 3. Colour System 4. Material System 5. Typography System 6. Platform Implementations

The Mosaic Design Language remains the primary authority.

External references provide inspiration rather than specification.

Internal References

MDL-001 — Vision

Provides:

- Companion philosophy
- Entertainment-first thinking
- Calm interaction
- Long-term product identity

Typography should reinforce the feeling that the user is accompanied rather than instructed.

MDL-002 — Principles

Provides:

- Content Leads
- Be A Companion
- Every Feature Earns Its Place
- Calm Interfaces

Typography should strengthen these principles through language, hierarchy and rhythm.

MDL-003 — Mental Model

Provides:

- World
- Focus
- Context
- Information

Typography communicates these concepts.

It should never redefine them.

MDL-004 — Interaction Model

Provides:

- Continuity
- Behaviour
- Reading flow
- User understanding

Editorial rhythm should reinforce behavioural continuity.

MDL-005 — Composition Model

Provides:

- Hero
- Hierarchy
- Priority
- Density
- Breathing Space

Typography expresses compositional hierarchy.

It should never compete with it.

MDS-001 — Design Token Architecture

Provides:

- Semantic Tokens
- Typography Roles
- Runtime Resolution

The Typography System consumes semantic roles rather than defining implementation values directly.

MDS-002 — Colour System

Provides:

- Semantic Colours
- Accessibility
- Runtime Atmosphere

Typography should remain readable regardless of atmospheric adaptation.

MDS-003 — Material System

Provides:

- Canvas
- Hero Material
- Overlay Material
- Physical hierarchy

Typography exists within Materials.

Its rhythm should feel physically related to them.

Future Specifications

The following specifications directly depend upon MDS-004.

- MDS-005 Motion System
- MDS-006 Composition Engine
- MDS-007 Tile Framework
- MDS-008 Component Library

These specifications should consume Typography.

They should not redefine editorial hierarchy.

Editorial Design

The Typography System draws heavily from editorial publishing rather than application dashboards.

Primary influences include:

- book typography
- magazine layout
- newspaper hierarchy
- long-form reading
- visual pacing

The objective is sustained understanding rather than information throughput.

Human Perception

Several characteristics of reading behaviour influenced the Typography System.

Examples include:

- eye movement
- reading rhythm
- line length
- paragraph pacing
- typographic hierarchy
- cognitive load

Typography should support natural reading behaviour rather than forcing users into interface scanning.

Accessibility

Accessibility is considered an architectural requirement.

The Typography System assumes:

```
[text]
Editorial Hierarchy

↓

Accessibility

↓

Runtime Resolution

↓

Presentation
```

This ordering ensures that readability always has higher priority than stylistic expression.

Responsive Reading

Unlike many responsive systems, Mosaic optimises typography according to:

- viewing distance
- reading behaviour
- editorial rhythm

rather than viewport size alone.

This distinction is expected to become increasingly important as Mosaic expands across:

- televisions
- tablets
- desktop
- mobile
- future display technologies.

Variable Typography

Variable Fonts are treated as implementation capability rather than design philosophy.

Typography should remain editorially stable regardless of whether the platform supports:

- variable fonts
- static font families
- future font technologies.

The Companion should always sound the same.

Platform Independence

Typography should remain conceptually identical across:

- Web
- Flutter
- SwiftUI
- Jetpack Compose
- Future rendering engines

Implementation differences should never alter editorial voice.

Mosaic-Specific Influences

The Typography System emerged directly from founder exploration.

Major architectural discoveries included:

- Typography should sound like a Companion rather than software.
- Editorial rhythm is more valuable than interface density.
- Hero Typography should introduce rather than advertise.
- Responsive Typography should optimise reading rather than layouts.
- Typography should quietly disappear behind understanding.

These ideas collectively define the typographic identity of Mosaic.

Relationship To The Companion

The Typography System represents the spoken voice of the Companion.

Conceptually.

```
[text]
Companion

↓

Editorial Language

↓

Typography

↓

Reading

↓

Understanding
```

This relationship should remain consistent throughout every Mosaic experience.

Typography is therefore considered behavioural as well as visual.

Normative References

Required reading before contributing to MDS-004.

- MDL-001 Vision
- MDL-002 Principles
- MDL-003 Mental Model
- MDL-004 Interaction Model
- MDL-005 Composition Model
- MDS-001 Design Token Architecture
- MDS-002 Colour System
- MDS-003 Material System

Together these specifications define the conceptual foundation of the Typography System.

Informative References

Future contributors may also wish to review:

- MDS-005 Motion System
- MDS-006 Composition Engine
- MDS-007 Tile Framework
- MDS-008 Component Library

These specifications describe how typography participates in interaction, composition and presentation.

Living Document

This reference list should remain intentionally concise.

References should only be introduced when they materially influence:

- editorial hierarchy,
- reading behaviour,
- runtime typography,
- implementation boundaries.

The objective is to preserve architectural reasoning rather than catalogue typography literature.

Completion

This concludes **MDS-004 — Typography System**.

The next specification in the Mosaic Design System is:

> ***MDS-005 — Motion System***

Where MDS-004 defines **how the Companion speaks**, MDS-005 defines **how the world moves**.

It formalises:

- Motion philosophy
- Behaviour-driven animation
- Temporal continuity
- Material motion
- Refraction movement
- Transition architecture
- Physics
- Accessibility-aware motion

- Runtime motion resolution

Motion is not decoration.

Within Mosaic, motion is the physical expression of understanding over time.